☕ Spring Boot

REST API — Complete Notes

Interview Preparation Guide

Real-World Project: Online Food Ordering System

📋 Topics Covered

MVC Architecture • REST Concepts • Annotations
Request Flow • CRUD APIs • Exception Handling
Validation • Response Formats • Interview Q&A

🍕 Project: FoodieExpress — Online Food Ordering App

Chapter 1: What is Spring Boot?

Think of Spring Boot as a ready-made kitchen. Instead of building everything from scratch (ovens, pots, pans), Spring Boot gives you everything pre-set so you can start cooking (building your app) right away!

🔷 Simple Analogy

Concept	Simple Explanation
Normal Spring	You buy raw materials & build the kitchen yourself
Spring Boot	You get a fully equipped kitchen — just start cooking!
REST API	A waiter who takes orders from clients & brings back food (data)
FoodieExpress App	Our real-world project we'll use throughout these notes

🔷 Key Benefits of Spring Boot

- Auto Configuration — No need to write XML configs
- Embedded Server — Tomcat runs inside, no separate install needed
- Starter Dependencies — One line adds everything you need (e.g., spring-boot-starter-web)
- Production Ready — Health checks, metrics built in

❓ **Interview Q:** What is the difference between Spring and Spring Boot?

✅ Spring = Framework. You configure everything manually (XML, annotations).
✅ Spring Boot = Spring + Auto-configuration + Embedded Server + Starters.
✅ Spring Boot reduces boilerplate code. In Spring, you need 50 lines to start. In Spring Boot, just 5.
✅ Example: In plain Spring, you configure DispatcherServlet manually. Spring Boot does it for you.

❓ **Interview Q:** What is @SpringBootApplication annotation?

✅ It is a combination of 3 annotations:
@Configuration — marks the class as a config source
@EnableAutoConfiguration — tells Spring Boot to auto-configure
@ComponentScan — scans the current package for Spring components
✅ You only need ONE annotation on your main class instead of three.

```
@SpringBootApplication
public class FoodieExpressApplication {
    public static void main(String[] args) {
        SpringApplication.run(FoodieExpressApplication.class, args);
    }
}
```

Chapter 2: MVC Architecture (Super Simple!)

MVC = Model + View + Controller. Think of a Restaurant 🍽️

🔷 Restaurant Analogy for MVC

MVC Layer	Restaurant Role
Model (M)	The Kitchen — stores and processes data (food preparation)
View (V)	The Plate / Menu — what the customer sees (JSON response in REST)
Controller (C)	The Waiter — takes request from customer & gives to kitchen
Client	The Customer — sends requests (your browser/mobile app)
Database	The Fridge/Storage — where all data is kept

🔷 MVC Request Flow — Step by Step

1. Client (Browser/App) sends HTTP Request → e.g., GET /api/restaurants
2. DispatcherServlet receives ALL requests (Front Controller pattern)
3. HandlerMapping finds the correct @RestController and method
4. Controller method executes — calls Service layer
5. Service layer contains business logic — calls Repository
6. Repository talks to Database and returns data
7. Data flows back: Repository → Service → Controller → JSON Response
8. Client receives JSON response ✅

🔷 Visual Flow Diagram

```
CLIENT (Browser / Mobile App)
↓ HTTP Request (GET /api/restaurants)
DISPATCHER SERVLET (Front Controller)
↓ Routes to correct Controller
@RestController (Controller Layer)
↓ calls
```

```
@Service (Business Logic Layer)
↓ calls
@Repository (Data Access Layer)
↓ queries
DATABASE (MySQL / PostgreSQL)
↑ JSON Response flows back up to Client ✅
```

❓ **Interview Q:** Explain MVC architecture in Spring Boot with an example.

✅ MVC = Model (data), View (response format), Controller (request handler).
✅ In REST APIs, the View is replaced by JSON response (no HTML pages).
✅ Flow: Request → DispatcherServlet → Controller → Service → Repository → DB
✅ Example: User hits GET /api/orders/5 → OrderController → OrderService → OrderRepository → DB
→ returns Order JSON back to user.
✅ Key Benefit: Separation of Concerns — each layer has ONE responsibility.

Chapter 3: What is REST API?

REST = Representational State Transfer. It is a set of rules for building web APIs. Think of REST as the rules of a game — both client and server must follow them.

🔷 REST Principles (6 Rules)

Principle	Meaning
1. Client-Server	Client and server are separate. Client doesn't care how server stores data.
2. Stateless	Each request is INDEPENDENT. Server does NOT remember previous requests.
3. Cacheable	Responses can be cached to improve performance.
4. Uniform Interface	Use standard HTTP methods (GET, POST, PUT, DELETE). Consistent URLs.
5. Layered System	Client doesn't know if it's talking to server directly or via proxy/load balancer.
6. Code on Demand	Server can send code (JavaScript) to client (optional rule).

🔷 HTTP Methods — The 4 CRUD Operations

HTTP Method	When to Use
GET	Read data — 'Get me all restaurants' → No body needed
POST	Create new data — 'Place a new order' → Send data in body
PUT	Update all fields — 'Update full user profile' → Send complete object
PATCH	Update specific field — 'Update only phone number' → Send only changed field
DELETE	Delete data — 'Cancel order #5' → Usually no body

❓ **Interview Q:** What is the difference between PUT and PATCH?

✅ PUT = Replace the ENTIRE resource. Send ALL fields even if only one changed.
✅ PATCH = Update PARTIAL resource. Send ONLY the fields that changed.
✅ Example — Updating a restaurant address:
PUT /restaurants/1 → { name: 'Pizza Palace', address: 'New St', phone: '99999', rating: 4.5 }
PATCH /restaurants/1 → { address: 'New St' } ← only address changed
✅ PATCH is more efficient when only one field needs updating.

❓ **Interview Q:** What is Stateless in REST? Why is it important?

✅ Stateless = Server does NOT store any session data between requests.
✅ Each request must contain ALL information needed to process it (e.g., auth token).
✅ Why important? Scalability — any server instance can handle any request (no sticky sessions).
✅ Example: When you call GET /api/orders, you send JWT token in header every time.
The server doesn't 'remember' you from the last request.

Chapter 4: Project Setup — FoodieExpress App

Our reference project: FoodieExpress — An Online Food Ordering System (like Swiggy/Zomato). We will build REST APIs for this app to understand every concept.

🔷 Project Package Structure

```
com.foodieexpress
├── FoodieExpressApplication.java    ← Main class
├── controller/
│   ├── RestaurantController.java
│   ├── OrderController.java
│   └── UserController.java
```

```
├── service/
│   ├── RestaurantService.java       ← Interface
│   └── impl/
│       └── RestaurantServiceImpl.java
├── repository/
│   └── RestaurantRepository.java    ← extends JpaRepository
├── model/  (Entity classes)
│   ├── Restaurant.java
│   ├── Order.java
│   └── User.java
├── dto/  (Data Transfer Objects)
│   ├── RestaurantDTO.java
│   └── OrderRequestDTO.java
└── exception/
    ├── GlobalExceptionHandler.java
    └── ResourceNotFoundException.java
```

🔷 pom.xml — Key Dependencies

```xml
<!-- Web (REST API) -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- JPA + Database -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- Validation -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- MySQL Driver -->
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
</dependency>

<!-- Lombok (Removes boilerplate getters/setters) -->
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
</dependency>
```

🔷 application.properties

```properties
# Database Config
spring.datasource.url=jdbc:mysql://localhost:3306/foodieexpress
spring.datasource.username=root
spring.datasource.password=password

# JPA Config
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

```
# Server Port
server.port=8080
```

Chapter 5: All Layers in Detail with Code

🔷 Layer 1: Model / Entity (The Data Blueprint)

Model = Blueprint of your data. It maps directly to a database table.

```
@Entity
@Table(name = "restaurants")
@Data                    // Lombok: auto generates getters/setters
@NoArgsConstructor
@AllArgsConstructor
public class Restaurant {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Name cannot be empty")
    @Column(nullable = false)
    private String name;

    private String address;
    private String cuisine;    // 'Italian', 'Indian', 'Chinese'
    private Double rating;
    private Boolean isOpen;
}
```

🔷 Layer 2: Repository (Database Operations)

Repository = Your database helper. Extends JpaRepository to get free CRUD methods.

```
@Repository
public interface RestaurantRepository extends JpaRepository<Restaurant, Long> {

    // Spring Data JPA auto-generates SQL for method names!
    List<Restaurant> findByCuisine(String cuisine);
    List<Restaurant> findByIsOpenTrue();
    List<Restaurant> findByRatingGreaterThanEqual(Double minRating);
    Optional<Restaurant> findByName(String name);

    // Custom JPQL query
    @Query("SELECT r FROM Restaurant r WHERE r.rating >= :rating AND r.isOpen = true")
    List<Restaurant> findOpenRestaurantsWithMinRating(@Param("rating") Double rating);
}
```

JpaRepository Method	SQL Equivalent
findAll()	SELECT * FROM restaurants
findById(1L)	SELECT * FROM restaurants WHERE id = 1
save(restaurant)	INSERT or UPDATE restaurant

JpaRepository Method	SQL Equivalent
deleteById(1L)	DELETE FROM restaurants WHERE id = 1
count()	SELECT COUNT(*) FROM restaurants
existsById(1L)	Check if record exists

❓ **Interview Q:** What is JpaRepository? What free methods does it give?

✅ JpaRepository is an interface from Spring Data JPA.
✅ It provides CRUD operations without writing any SQL.
✅ Free methods: save(), findById(), findAll(), deleteById(), count(), existsById()
✅ You can also create custom methods by just naming them: findByName(), findByAge()
✅ Spring automatically generates the SQL query from the method name!

🔷 Layer 3: Service (Business Logic)

Service = The brain. Contains all business rules. Controller calls Service, never the Repository directly.

```java
// Interface
public interface RestaurantService {
    List<Restaurant> getAllRestaurants();
    Restaurant getById(Long id);
    Restaurant createRestaurant(Restaurant restaurant);
    Restaurant updateRestaurant(Long id, Restaurant restaurant);
    void deleteRestaurant(Long id);
}

// Implementation
@Service
public class RestaurantServiceImpl implements RestaurantService {

    @Autowired
    private RestaurantRepository repository;

    @Override
    public List<Restaurant> getAllRestaurants() {
        return repository.findAll();
    }

    @Override
    public Restaurant getById(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException(
                "Restaurant not found with id: " + id));
    }

    @Override
    public Restaurant createRestaurant(Restaurant restaurant) {
        // Business rule: rating must be between 0-5
        if (restaurant.getRating() < 0 || restaurant.getRating() > 5) {
            throw new IllegalArgumentException("Rating must be between 0 and 5");
        }
        return repository.save(restaurant);
    }
```

```
    @Override
    public void deleteRestaurant(Long id) {
        Restaurant existing = getById(id);  // throws if not found
        repository.delete(existing);
    }
}
```

🔷 Layer 4: Controller (Request Handler)

Controller = The waiter. Receives HTTP requests and returns HTTP responses. Should have NO business logic.

```
@RestController
@RequestMapping("/api/restaurants")
public class RestaurantController {

    @Autowired
    private RestaurantService service;

    // GET all restaurants
    @GetMapping
    public ResponseEntity<List<Restaurant>> getAllRestaurants() {
        return ResponseEntity.ok(service.getAllRestaurants());
    }

    // GET restaurant by ID
    @GetMapping("/{id}")
    public ResponseEntity<Restaurant> getById(@PathVariable Long id) {
        return ResponseEntity.ok(service.getById(id));
    }

    // POST - create new restaurant
    @PostMapping
    public ResponseEntity<Restaurant> create(@Valid @RequestBody Restaurant restaurant) {
        Restaurant saved = service.createRestaurant(restaurant);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    // PUT - full update
    @PutMapping("/{id}")
    public ResponseEntity<Restaurant> update(
            @PathVariable Long id,
            @Valid @RequestBody Restaurant restaurant) {
        return ResponseEntity.ok(service.updateRestaurant(id, restaurant));
    }

    // DELETE
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.deleteRestaurant(id);
        return ResponseEntity.noContent().build();  // 204
    }

    // GET with query params - search by cuisine
    @GetMapping("/search")
    public ResponseEntity<List<Restaurant>> search(
            @RequestParam String cuisine,
            @RequestParam(defaultValue = "0.0") Double minRating) {
        return ResponseEntity.ok(service.searchByCuisineAndRating(cuisine, minRating));
    }
}
```

Chapter 6: Key Annotations — Quick Reference

🔷 Controller Annotations

Annotation	Purpose
@RestController	@Controller + @ResponseBody. Returns JSON/XML directly (no View resolver)
@Controller	Returns View name (for MVC with HTML pages — not used in REST)
@RequestMapping	Base URL mapping for the entire controller class
@GetMapping	Handles HTTP GET request — for fetching data
@PostMapping	Handles HTTP POST request — for creating data
@PutMapping	Handles HTTP PUT request — for full update
@PatchMapping	Handles HTTP PATCH request — for partial update
@DeleteMapping	Handles HTTP DELETE request — for deleting data

🔷 Parameter Annotations

Annotation	When to Use
@PathVariable	Extracts value from URL path: /restaurants/{id} → @PathVariable Long id
@RequestParam	Extracts query param: /search?cuisine=Italian → @RequestParam String cuisine
@RequestBody	Maps request body JSON to Java object: { 'name': 'Pizza Palace' } → Restaurant object
@RequestHeader	Extracts HTTP header value: Authorization: Bearer token → @RequestHeader String auth
@Valid	Triggers validation on the @RequestBody object (checks @NotBlank, @Size, etc.)

🔷 Stereotype Annotations

Annotation	Purpose
@Component	Generic Spring bean — any class you want Spring to manage
@Service	@Component + Semantic: marks business logic layer classes
@Repository	@Component + Exception translation for DB errors
@Controller / @RestController	@Component + Semantic: marks web request handlers

Annotation	Purpose
@Autowired	Auto-inject a Spring bean (dependency injection)
@Bean	Manually declare a Spring bean inside @Configuration class
@Configuration	Marks class as configuration source (like old XML config)

❓ **Interview Q:** Difference between @Controller and @RestController?

✅ @Controller: Returns a View name (HTML page). Used in MVC web apps.
✅ @RestController: Returns data directly (JSON/XML). = @Controller + @ResponseBody
✅ In REST APIs, ALWAYS use @RestController.
✅ @ResponseBody tells Spring: don't look for a View, just write the return value to HTTP response.

❓ **Interview Q:** What is @PathVariable vs @RequestParam?

✅ @PathVariable — value is part of the URL PATH
Example: GET /restaurants/5 → @GetMapping('/{id}') @PathVariable Long id → id = 5

✅ @RequestParam — value comes after '?' in URL (query string)
Example: GET /restaurants/search?cuisine=Italian → @RequestParam String cuisine → cuisine = 'Italian'

✅ Rule of thumb: ID in path → PathVariable. Filters/search → RequestParam

Chapter 7: HTTP Status Codes — Must Know

Status Code	When to Return
200 OK	Request succeeded. Used for GET, PUT, PATCH success.
201 Created	New resource created successfully. Used for POST success.
204 No Content	Request succeeded but no body returned. Used for DELETE.
400 Bad Request	Client sent invalid data. Validation failed.
401 Unauthorized	Not logged in. Missing or invalid token.
403 Forbidden	Logged in but no permission to access this resource.
404 Not Found	Resource doesn't exist. e.g., Restaurant with id=999 not found.
409 Conflict	Resource already exists. e.g., Email already registered.
422 Unprocessable	Data format is right but semantic validation failed.

Status Code	When to Return
500 Internal Server Error	Something broke on the server side (our bug).

🔷 ResponseEntity — Proper Way to Return Status

```
// Return 200 OK with body
return ResponseEntity.ok(restaurant);

// Return 201 Created with body
return ResponseEntity.status(HttpStatus.CREATED).body(savedRestaurant);

// Return 204 No Content (for DELETE)
return ResponseEntity.noContent().build();

// Return 404 Not Found
return ResponseEntity.notFound().build();

// Return 400 with custom message
return ResponseEntity.badRequest().body("Invalid input");

// Return custom status
return ResponseEntity.status(HttpStatus.CONFLICT).body("Restaurant already exists");
```

❓ **Interview Q:** What is ResponseEntity? Why use it over direct return?

✅ ResponseEntity wraps your response body AND allows you to set the HTTP status code.
✅ Without it: Spring returns 200 OK by default — even for POST (should be 201).
✅ With it: You have full control over status, headers, and body.
✅ Best Practice: Always use ResponseEntity<T> as return type in controllers.
✅ Generic type T ensures type safety: ResponseEntity<Restaurant>, ResponseEntity<List<Restaurant>>

Chapter 8: Exception Handling

Without proper exception handling, users see ugly 500 errors. We need to return clean, meaningful error responses.

🔷 Step 1: Custom Exception Class

```
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

🔷 Step 2: Error Response DTO

```
@Data
@AllArgsConstructor
public class ErrorResponse {
    private int status;
    private String message;
    private LocalDateTime timestamp;
}
```

🔷 Step 3: Global Exception Handler

```
@RestControllerAdvice       // handles exceptions across ALL controllers
public class GlobalExceptionHandler {

    // Handle our custom exception
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
        ErrorResponse error = new ErrorResponse(
            404, ex.getMessage(), LocalDateTime.now());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    // Handle validation errors
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(
            MethodArgumentNotValidException ex) {
        String message = ex.getBindingResult().getFieldErrors()
            .stream()
            .map(e -> e.getField() + ": " + e.getDefaultMessage())
            .collect(Collectors.joining(", "));
        return ResponseEntity.badRequest()
            .body(new ErrorResponse(400, message, LocalDateTime.now()));
    }

    // Handle ALL other exceptions
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneral(Exception ex) {
        ErrorResponse error = new ErrorResponse(
            500, "Something went wrong", LocalDateTime.now());
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
    }
}
```

🔷 Error Response Output (Clean JSON)

```
{
  "status": 404,
  "message": "Restaurant not found with id: 999",
  "timestamp": "2024-01-15T10:30:45"
}
```

❓ **Interview Q:** What is @ControllerAdvice vs @RestControllerAdvice?

✅ @ControllerAdvice = Global exception handler for MVC controllers (returns View/HTML).
✅ @RestControllerAdvice = @ControllerAdvice + @ResponseBody → returns JSON.
✅ In REST APIs, always use @RestControllerAdvice.
✅ @ExceptionHandler inside it handles specific exception types.
✅ Without this: each controller would need try-catch everywhere (messy!).

Chapter 9: Input Validation

🔷 Validation Annotations on Model

```
@Entity
public class Restaurant {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Restaurant name is required")
    @Size(min = 2, max = 100, message = "Name must be 2-100 characters")
    private String name;

    @NotBlank(message = "Address cannot be empty")
    private String address;

    @Min(value = 0, message = "Rating cannot be negative")
    @Max(value = 5, message = "Rating cannot exceed 5")
    private Double rating;

    @Email(message = "Invalid email format")
    private String contactEmail;

    @Pattern(regexp = "^[0-9]{10}$", message = "Phone must be 10 digits")
    private String phone;
}
```

Annotation	Rule
@NotNull	Field cannot be null (but can be empty string)
@NotBlank	Field cannot be null, empty, or whitespace only
@NotEmpty	Field cannot be null or empty (allows whitespace)
@Size(min,max)	String/Collection length must be within range
@Min(value)	Number must be >= value
@Max(value)	Number must be <= value
@Email	Must be valid email format
@Pattern(regexp)	Must match regex pattern
@Positive	Number must be > 0
@Future	Date must be in the future
@Past	Date must be in the past

💡 **Tip:** Add @Valid on @RequestBody in the controller to trigger validation. Without @Valid, the annotations on the model are ignored!

Chapter 10: DTO Pattern (Data Transfer Object)

DTO = What you expose to clients. Entity = What you store in DB. NEVER expose your Entity directly to API consumers.

🔷 Why Use DTO?

- Security: Don't expose password, internal IDs to clients
- Flexibility: Request DTO can differ from Response DTO
- API Stability: Change DB schema without breaking API contracts
- Validation: Validation rules on DTO, not entity

🔷 FoodieExpress DTO Example

```
// Request DTO — what client sends when placing an order
@Data
public class OrderRequestDTO {
    @NotNull
    private Long restaurantId;

    @NotEmpty(message = "Order must have at least one item")
    private List<Long> menuItemIds;

    private String deliveryAddress;
}

// Response DTO — what we send back to client
@Data
public class OrderResponseDTO {
    private Long orderId;
    private String restaurantName;
    private List<String> items;
    private Double totalAmount;
    private String status;         // 'PLACED', 'PREPARING', 'DELIVERED'
    private LocalDateTime orderTime;
    // Note: No internal DB fields like createdBy, updatedAt, etc.
}
```

❓ **Interview Q:** What is DTO? Why not use Entity directly in API?

✅ DTO = Data Transfer Object. A plain Java class (no JPA annotations).
✅ Entity is the DB representation. DTO is the API representation.

✅ Problem with Entity directly:
1. Security: You may expose password field accidentally
2. Circular references: @OneToMany causes infinite JSON loop
3. Tight coupling: DB change breaks API

✅ DTO solves all 3 problems. Use ModelMapper or MapStruct for conversion.

Chapter 11: Complete CRUD API — Order Management

Let's trace a complete order placement flow in FoodieExpress, from HTTP request to database and back.

🔷 API Endpoints — Order Module

Endpoint	Description
POST /api/orders	Place a new order
GET /api/orders	Get all orders (admin)
GET /api/orders/{id}	Get specific order by ID
GET /api/orders/user/{userId}	Get all orders for a user
PATCH /api/orders/{id}/status	Update order status (e.g., DELIVERING)
DELETE /api/orders/{id}	Cancel an order

🔷 Complete Flow: POST /api/orders (Place an Order)

9. Client sends: POST /api/orders
10. with body: { restaurantId: 1, menuItemIds: [3, 5], deliveryAddress: '123 MG Road' }
11. OrderController.createOrder(@Valid @RequestBody OrderRequestDTO dto) is called
12. Controller calls orderService.placeOrder(dto)
13. Service validates: restaurant exists? items belong to that restaurant?
14. Service calculates total amount, creates Order entity
15. orderRepository.save(order) → INSERT into orders table
16. Service maps Order entity → OrderResponseDTO
17. Controller returns ResponseEntity.status(201).body(responseDTO)
18. Client receives 201 Created + order details JSON ✅

🔷 The Complete Controller Code

```
@RestController
@RequestMapping("/api/orders")
public class OrderController {
```

```
    @Autowired
    private OrderService orderService;

    @PostMapping
    public ResponseEntity<OrderResponseDTO> placeOrder(
            @Valid @RequestBody OrderRequestDTO dto) {
        OrderResponseDTO response = orderService.placeOrder(dto);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    @GetMapping("/{id}")
    public ResponseEntity<OrderResponseDTO> getOrder(@PathVariable Long id) {
        return ResponseEntity.ok(orderService.getOrderById(id));
    }

    @GetMapping("/user/{userId}")
    public ResponseEntity<List<OrderResponseDTO>> getUserOrders(
            @PathVariable Long userId) {
        return ResponseEntity.ok(orderService.getOrdersByUser(userId));
    }

    @PatchMapping("/{id}/status")
    public ResponseEntity<OrderResponseDTO> updateStatus(
            @PathVariable Long id,
            @RequestParam String status) {
        return ResponseEntity.ok(orderService.updateOrderStatus(id, status));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> cancelOrder(@PathVariable Long id) {
        orderService.cancelOrder(id);
        return ResponseEntity.noContent().build();
    }
}
```

Chapter 12: Dependency Injection (DI)

DI = Instead of creating objects yourself (new Service()), Spring creates and gives them to you. Like a delivery boy who brings you what you need — you don't go to the shop.

🔷 3 Ways to Inject Dependencies

Type	Notes
Field Injection (@Autowired on field)	Easiest to write but HARD to test. Avoid in production.
Constructor Injection (RECOMMENDED)	Best practice. Testable. Makes dependencies explicit.
Setter Injection	Rare. Used for optional dependencies.

```
// ❌ AVOID: Field Injection — hard to unit test
@Service
public class OrderServiceImpl {
```

```
    @Autowired
    private OrderRepository repository;  // Spring injects this
}

// ✅ PREFERRED: Constructor Injection
@Service
public class OrderServiceImpl {
    private final OrderRepository repository;

    // @Autowired is optional when there's only ONE constructor (Spring 4.3+)
    public OrderServiceImpl(OrderRepository repository) {
        this.repository = repository;
    }
}

// ✅ With Lombok @RequiredArgsConstructor (cleanest!)
@Service
@RequiredArgsConstructor
public class OrderServiceImpl {
    private final OrderRepository repository;  // final = required constructor arg
}
```

❓ **Interview Q:** What is Dependency Injection? What are its types?

✅ DI = Design pattern where Spring creates and manages object dependencies.
✅ You don't do 'new Service()' — Spring does it and injects it where needed.
✅ 3 Types: Field Injection, Constructor Injection (best), Setter Injection.
✅ Why Constructor Injection is best:
- Makes dependencies visible and explicit
- Easy to test with mocks (just pass mock in constructor)
- Fields can be 'final' (immutable, thread-safe)

Chapter 13: Top Interview Q&A — Quick Revision

❓ **Interview Q:** What happens internally when you start a Spring Boot app?

1. SpringApplication.run() is called
2. ApplicationContext is created (IoC container)
3. @ComponentScan scans for @Component, @Service, @Repository, @Controller
4. Auto-configuration kicks in (based on classpath JARs)
5. Embedded Tomcat starts on port 8080
6. DispatcherServlet is registered to handle all HTTP requests
7. Application is READY to receive requests ✅

❓ **Interview Q:** What is IoC Container? What is ApplicationContext?

✅ IoC = Inversion of Control. Instead of YOU creating objects, Spring creates them.
✅ IoC Container = Spring's object factory. Creates, manages, and destroys beans.
✅ ApplicationContext = Main IoC Container interface in Spring Boot.
✅ It holds all beans (singletons by default) and injects them via DI.
✅ BeanFactory is the parent. ApplicationContext adds more features (events, i18n).

❓ **Interview Q:** What is the Bean lifecycle in Spring?

1. Instantiate → Spring creates the object using constructor
2. Populate Properties → @Autowired dependencies injected
3. @PostConstruct → Custom init method runs (if defined)
4. Bean is READY for use
5. @PreDestroy → Custom cleanup method runs on shutdown
6. Bean is Destroyed

❓ **Interview Q:** What are Bean Scopes in Spring?

✅ Singleton (DEFAULT) — ONE instance per Spring container. Shared everywhere.
✅ Prototype — NEW instance every time the bean is requested.
✅ Request — ONE instance per HTTP request (web apps only).
✅ Session — ONE instance per HTTP session (web apps only).
✅ 99% of the time you use Singleton. Prototype is for stateful beans.

❓ **Interview Q:** How does Spring Boot Auto-Configuration work?

✅ Spring Boot checks what JARs are on classpath (spring.factories / AutoConfiguration.imports).
✅ If spring-boot-starter-web is present → auto-configures DispatcherServlet, Tomcat, Jackson.
✅ If spring-boot-starter-data-jpa is present → auto-configures DataSource, EntityManager.
✅ @ConditionalOnClass, @ConditionalOnMissingBean — configures ONLY if class exists / bean missing.
✅ You can OVERRIDE auto-config by providing your own @Bean.

❓ **Interview Q:** What is @Transactional? When to use it?

✅ @Transactional ensures all DB operations inside a method succeed or all fail (ACID).
✅ If any exception occurs → all changes are rolled back automatically.

✅ Use on Service methods that do multiple DB operations together.
✅ Example: Placing an order: (1) Deduct stock (2) Create order (3) Charge payment
→ If step 3 fails, steps 1 and 2 should also rollback!
✅ Only marks rollback for RuntimeException by default. Use rollbackFor=Exception.class for all.

❓ **Interview Q:** What is the difference between @Component, @Service, @Repository?

✅ All 3 are specializations of @Component — they all make Spring create a bean.
✅ @Component → generic bean (utility classes, helpers)
✅ @Service → business logic layer (semantic meaning, no extra behavior)
✅ @Repository → data access layer + Spring converts JPA exceptions to DataAccessException
✅ Use the right one for the right layer — makes code more readable and self-documenting.

❓ **Interview Q:** How do you handle CORS in Spring Boot?

✅ CORS = Cross-Origin Resource Sharing. Browser blocks requests from different domains by default.
✅ Option 1: @CrossOrigin on controller: @CrossOrigin(origins = 'http://localhost:3000')
✅ Option 2: Global config in WebMvcConfigurer:

```
registry.addMapping('/api/**').allowedOrigins('*').allowedMethods('GET','POST','PUT','DELETE')
```

✅ In production: specify exact origins, not '*' (security risk).

❓ **Interview Q:** What is Lombok? How does it help?

✅ Lombok = Java library that auto-generates boilerplate code at compile time.
✅ @Data = @Getter + @Setter + @ToString + @EqualsAndHashCode + @RequiredArgsConstructor
✅ @NoArgsConstructor = no-arg constructor
✅ @AllArgsConstructor = constructor with all fields
✅ @Builder = Builder pattern for object creation
✅ @Slf4j = auto-creates 'log' logger field
✅ Saves 100+ lines per class in large projects!

Chapter 14: Final Cheat Sheet

🔷 REST API Design Rules (Best Practices)

- Use nouns in URLs, NOT verbs: /restaurants ✅ /getRestaurants ❌
- Use plural nouns: /restaurants ✅ /restaurant ❌
- Use HTTP method for the action: GET=read, POST=create, PUT=update, DELETE=delete
- Use nested URLs for relationships: GET /restaurants/1/menus
- Return proper HTTP status codes: 201 for POST, 204 for DELETE, 404 for not found
- Validate all inputs with @Valid + Bean Validation annotations
- Handle exceptions globally with @RestControllerAdvice
- Use DTOs — never expose your entity directly
- Use ResponseEntity for full control over response
- Version your API: /api/v1/restaurants

🔷 FoodieExpress — All APIs at a Glance

API Endpoint	Purpose
GET /api/restaurants	List all restaurants
GET /api/restaurants/{id}	Get restaurant by ID
GET /api/restaurants/search?cuisine=Indian	Search by cuisine
POST /api/restaurants	Add new restaurant (admin)
PUT /api/restaurants/{id}	Full update restaurant
DELETE /api/restaurants/{id}	Remove restaurant
POST /api/orders	Place new food order
GET /api/orders/user/{userId}	Get user's order history
PATCH /api/orders/{id}/status	Update order status
DELETE /api/orders/{id}	Cancel an order
POST /api/users/register	Register new user
POST /api/users/login	User login → returns JWT

🔷 Spring Boot Layers — One-Line Summary

Component	One-Line Purpose
@Entity / Model	Maps to DB table. Has fields, JPA annotations.
@Repository	Extends JpaRepository. CRUD without SQL.
@Service	Business logic. Called by Controller. Calls Repository.

Component	One-Line Purpose
@RestController	Handles HTTP. Calls Service. Returns JSON.
DTO	Data shape for API (not DB). Hides internal fields.
@ExceptionHandler	Catches exceptions, returns clean error JSON.
@Valid	Triggers Bean Validation on incoming request body.
ResponseEntity<T>	Wrapper for HTTP response with status + body.

🎯 Interview Tip

Always explain with an example.

Connect every answer to the FoodieExpress real-world project.

"In my project, we had restaurants, orders, and users..."

Interviewers love candidates who think in real-world terms! ✅

All the best for your Interview! 🚀

Spring Boot REST API — FoodieExpress Edition